

SPL-1 Proposal Report

Group-11

Virtual File System Simulator

Course: SWE 4304 Software Project Lab 1

Supervisor: Samnun Azfar

Junior Lecturer

IUT,CSE

Team Members:

- ❖ Zuhaaer Mustafa Sadab (230042132)
- ❖ Ashfaque Sadad (230042102)
- ❖ Hasan Mobarak Mahi (230042129)

Project Overview

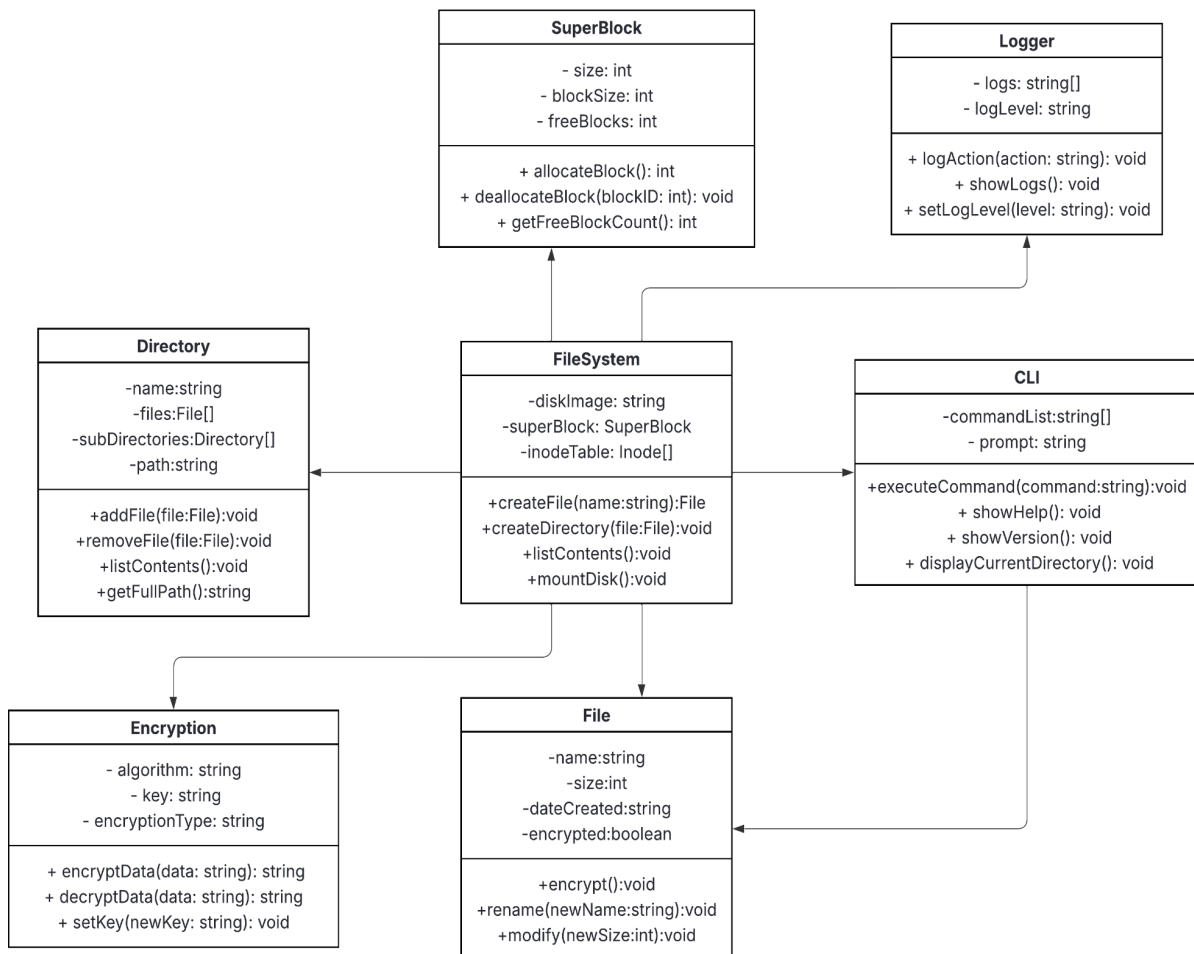
The **Virtual File System Simulator** aims to replicate basic file system operations like creating, deleting, moving, and renaming files, as well as navigating directories—all through a **command-line interface (CLI)**. Developed in **C++**, this system will interact with a **virtual disk file** (`disk.img`), simulating the core functions of a real file system. By using object-oriented principles, the project provides a hands-on way to explore file management concepts, file metadata, and space allocation, offering a deeper understanding of how file systems work behind the scenes.

Motivation Behind the Project

The motivation behind this project is to explore the inner workings of file systems, which we use daily without fully understanding their underlying processes. By simulating a file system through a **command-line interface (CLI)**, the project provides an opportunity to dive into key concepts such as file operations, directory structures, and file metadata management. It offers a practical way to learn how operating systems manage files, while also honing skills in **C++** programming, **object-oriented design**, and version control with **Git**.

UML Diagram

The UML diagram shows the main components of the Virtual File System Simulator: The CommandLineInterface for user input, FileSystem for managing operations, VirtualDisk for storage, DirectoryEntry for file organization, and SecurityModule for encryption.



Key Features

1. Directory Management

- Users can create, delete, and navigate through directories, mimicking standard file system behavior.
- Supports hierarchical directory structures to organize files efficiently.

2. File Operations

- Allows for file creation, renaming, moving, and deletion through simple CLI commands.
- Supports file metadata management such as file names, sizes, and timestamps.

3. File Encryption

- Basic file encryption using algorithms like Caesar Cipher or XOR for secure file handling.
- Ensures that files can be password-protected .

4. Command-Line Interface (CLI)

- Provides a text-based interface for managing files and directories, making the system lightweight and efficient.
- Custom CLI commands are implemented for file operations, improving user control and interaction.

5. Logging and Auditing

- Tracks all file system operations (e.g., file creation, deletion) for debugging and auditing purposes.
- Logs help ensure transparency and facilitate error handling throughout the system.

6. Directory Listing and Sorting

- Allows users to list the contents of a directory, displaying files and subdirectories.
- Supports sorting files by name, size, or modification date, making it easier to organize and find files.

Tools & Technologies

1. Programming Language: C++

- C++ is used for system implementation, offering high performance and low-level memory management.

2. Object-Oriented Programming (OOP) Principles

- OOP principles like classes, inheritance, and encapsulation help structure the project into modular, reusable parts.

3. Data Structures

- **Trees** represent hierarchical directory structures, enabling efficient file management and traversal.
- **Linked Lists** are used to dynamically store and manage files within directories, offering flexibility in file operations.

4. Version Control: Git

- Git tracks changes in the code, enabling version control and collaboration.

5. Virtual Storage: disk.img

- `disk.img` simulates a virtual disk, storing files, directories, and metadata.
- It mimics a real disk's storage management, allowing file operations without interacting with physical hardware.

Proposed Timeline:

Weeks	Tasks
Week 1	Idea generation and brainstorming
Week 2	Research on file system structure
Week 3	Project planning
Week 4	Finalize project scope
Week 5	Disk Structure Design
Week 6	File System Structure & Metadata
Week 7	Directory System Implementation
Week 8	Directory System Implementation
Week 9	CLI Development
Week 10	File Encryption Integration
Week 11	Logging and Search Features
Week 12	Data Integrity & Security Integration
Week 13	Full System Integration & Testing

Action Taken Based on Feedback

1. **Slide Quality:** Simplified slides by reducing text and using concise bullet points for clarity and readability.
2. **Weekly Testing:** Updated timeline to include weekly testing to validate each feature as it is developed.
3. **Class Diagram:** Corrected the class diagram to accurately represent system components and relationships.
4. **Timeline Adjustments:** Integrated continuous testing and documentation throughout the project, rather than at the end.
5. **Concise Presentation:** Reduced paragraph text and replaced with bullet points for a more streamlined and engaging presentation.